



JAY TECH SOLUTIONS

JTS GO Tutorials

Learn by Doing — Twelve Step-by-Step Programs from
Hello, World! to Machine Learning & Web Servers

HANDS-ON EDITION

VERSION 0.9.3-BETA

WRITE YOUR FIRST PROGRAM · VARIABLES & MATH · CONDITIONALS & LOOPS

FUNCTIONS · LISTS, DICTS & SETS · CLASSES · FILES & JSON

ERROR HANDLING · MACHINE LEARNING · WEB DEVELOPMENT

ASWINJAY — CEO, JAY TECH SOLUTIONS

COMPANION VOLUME TO THE JTS GO PROGRAMMING LANGUAGE

COPYRIGHT © 2026 JAY TECH SOLUTIONS · ALL RIGHTS RESERVED

JTS GO Tutorials

Learn by Doing — Twelve Step-by-Step Programs

Version 0.9.3-beta · Hands-On Edition

ASWINJAY

CEO, Jay Tech Solutions

JTS GO Tutorials

Learn by Doing — Twelve Step-by-Step Programs

First Edition · Hands-On Edition · 2026

Copyright © 2026 Jay Tech Solutions.

All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

JTS GO, JTS VM, JTS IDE and the JTS GO logo are trademarks of Jay Tech Solutions. Windows is a registered trademark of Microsoft Corporation. All other trademarks are the property of their respective owners.

Every tutorial in this book has been executed and verified against the official JTS GO interpreter (Version 0.9.3-beta).

Published by Jay Tech Solutions

<https://github.com/jayaswinjay-web/jtsdk>

How to Use This Book

This is the hands-on companion to *The JTS GO Programming Language*. Where the reference book explains the language topic by topic, this book teaches it the way most people actually learn: by running real programs.

There are twelve tutorials, one per chapter, and they build on each other like a ladder. Tutorial 1 is a three-line “Hello, World!” program; by Tutorial 12 you will have touched machine learning and web servers. Each chapter presents one complete, runnable program together with a short explanation and a *Challenge* at the end. Do not skip the challenges — they are the whole point of a learn-by-doing book.

Every tutorial is also a file in the repository's `tutorials/` folder. If you installed JTS GO through npm, the folder is already on your machine inside the installation directory. Run any tutorial from any folder:

```
jts tutorials/01_hello.jts
```

The scrolls some tutorials use (`math`, `strings`, `lists`, `sets`, `json`, `fs`) resolve automatically no matter which folder you are in, so you can run every example exactly as printed.

— Aswinjay
CEO, Jay Tech Solutions

Contents

How to Use This Book	3
.....	
Tutorial 1: Hello, World!	5
.....	
Tutorial 2: Variables and Math	6
.....	
Tutorial 3: Conditionals	7
.....	
Tutorial 4: Loops	8
.....	
Tutorial 5: Strings	9
.....	
Tutorial 6: Functions	10
.....	
Tutorial 7: Lists	11
.....	
Tutorial 8: Dictionaries and Sets	12
.....	
Tutorial 9: Classes and Inheritance	13
.....	
Tutorial 10: Files and JSON	15
.....	
Tutorial 11: Errors and try/catch/finally	16
.....	
Tutorial 12: Machine Learning and Web	17
.....	

Tutorial 1: Hello, World!

Source file: `tutorials/01_hello.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Write your first program with `say`.

Run it: `jts tutorials/01_hello.jts`

```
say("Hello, World!")
say("Welcome to the JTS GO tutorial!")
```

Challenge: change the greeting to your own name.

Tutorial 2: Variables and Math

Source file: [tutorials/02_variables_math.jts](#) — a complete, runnable program. Every tutorial in this book is a file in the repository's [tutorials/](#) folder.

Goal: Store values in variables and do arithmetic.

JTS is dynamically typed: `x = 42`. Optional type hints: `int age = 25`.

Operators: `+` `-` `*` `/` `%` and bitwise `&` `|` `^` `~` `<<` `>>`

`str(x)` converts numbers so you can join them with `+`.

```
name = "Alex"
age = 7
say("Hi, " + name + "!")
say("In 3 years you will be " + str(age + 3))

pi = 3.14159
say("Pi doubled: " + str(pi * 2))
say("Remainder of 17 / 5: " + str(17 % 5))

# Challenge: compute and print the area of a circle with radius 4.
```

Tutorial 3: Conditionals

Source file: `tutorials/03_conditionals.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Make decisions with `if` / `elif` / `else`.

Comparisons: `==` `!=` `<` `<=` `>` `>=` . Logic: `and` `or` `not` . Membership: `of` .

```
score = 85
if score >= 90
    say("Grade: A")
elif score >= 80
    say("Grade: B")
elif score >= 70
    say("Grade: C")
else
    say("Grade: F")
end

x = 7
if x > 0 and x % 2 == 0
    say("positive and even")
elif x > 0
    say("positive and odd")
end

primes = {2, 3, 5, 7}
if 7 of primes
    say("7 is prime")
end
```

Challenge: print "hot", "warm", or "cold" based on a temperature variable.

Tutorial 4: Loops

Source file: `tutorials/04_loops.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Repeat work with `for` and `while`.

IMPORTANT: `for i of 0 to 10` is INCLUSIVE: $i = 0, 1, \dots, 10$.

`for x of items` iterates a list. `while cond` loops until false.

```
for i of 1 to 5
  say("Counting: " + str(i))
end

items = ["apple", "banana", "cherry"]
for fruit of items
  say("I like " + fruit)
end

n = 0
while n < 3
  n = n + 1
  if n == 2
    continue
  end
  say("while n = " + str(n))
end

for i of 1 to 10
  if i == 4
    break
  end
  say("until break: " + str(i))
end
```

Challenge: print the first 5 even numbers using a Loop.

Tutorial 5: Strings

Source file: `tutorials/05_strings.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Slice, search, and transform text.

Strings have methods: `upper()`, `lower()`, `trim()`, `contains(s)`,

`replace(a, b)`, `substring(s, e)`, `starts_with(s)`, `split(sep)`, `join(list)`.

The `strings` scroll adds more helpers.

```
greeting = " Hello, JTS GO! "  
say("trimmed: [" + greeting.trim() + "]")  
say("upper: " + greeting.upper())  
say("lower: " + greeting.lower())  
say("contains 'JTS': " + str(greeting.contains("JTS")))  
say("starts with 'Hi': " + str(greeting.starts_with("Hi")))  
say("replaced: " + greeting.replace("JTS", "World"))  
  
text = "a,b,c,d"  
parts = text.split(",")  
say("split: " + str(parts))  
say("joined: " + "-".join(parts))  
say("length: " + str(len(text)))  
  
bring strings  
say("reversed: " + strings.reverse("hello"))  
say("repeat: " + strings.repeat("ab", 3))  
say("is palindrome: " + str(strings.is_palindrome("racecar")))  
say("word count: " + str(strings.count_words("the quick brown fox")))  
say("words: " + str(strings.words("one two three")))  
  
# Challenge: print your name, then reverse it.
```

Tutorial 6: Functions

Source file: `tutorials/06_functions.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Reuse code with `func`, parameters, `return`, and recursion.

Define: `func name(params) ... end`. Call: `name(args)`.

A function without `return` returns `void`.

```
func greet(name)
    say("Hello, " + name + "!")
end

func add(a, b)
    return a + b
end

func is_even(n)
    if n % 2 == 0
        return true
    end
    return false
end

func factorial(n)
    if n <= 1
        return 1
    end
    return n * factorial(n - 1)
end

greet("Alex")
say("add(3, 4) = " + str(add(3, 4)))
say("is_even(10) = " + str(is_even(10)))
say("is_even(7) = " + str(is_even(7)))
say("factorial(5) = " + str(factorial(5)))

# Functions calling functions.
func square(x)
    return x * x
end

func sum_of_squares(a, b)
    return square(a) + square(b)
end

say("sum_of_squares(3, 4) = " + str(sum_of_squares(3, 4)))

# Challenge: write a function that counts down from n to 1.
```

Tutorial 7: Lists

Source file: `tutorials/07_lists.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Store many values and transform them.

Indexing starts at 0: `items[0]`. Append with `append(list, v)` or `list.append(v)`.

Iterate with `for x of items`. Built-ins: `len()`, `sort()`, `pop()`, `reverse()`.

```
fruits = ["apple", "banana", "cherry"]
say("first: " + fruits[0])
say("last: " + fruits[2])
say("length: " + str(len(fruits)))

fruits.append("date")
say("after append: " + str(fruits))
fruits.pop()
say("after pop: " + str(fruits))

numbers = [5, 3, 9, 1, 7]
numbers.sort()
say("sorted: " + str(numbers))

total = 0
for n of numbers
  total = total + n
end
say("sum: " + str(total))

bring lists
nested = [1, [2, 3], [4, [5, 6]]]
say("flattened: " + str(lists.flatten(nested)))
say("unique of [1,2,2,3,3,3]: " + str(lists.unique([1, 2, 2, 3, 3, 3])))
say("reversed [1,2,3,4]: " + str(lists.reversed([1, 2, 3, 4])))
say("first [9,8,7]: " + str(lists.first([9, 8, 7])))
say("last [9,8,7]: " + str(lists.last([9, 8, 7])))

# Challenge: find the largest value in a list using a loop.
```

Tutorial 8: Dictionaries and Sets

Source file: [tutorials/08_dicts_sets.jts](#) — a complete, runnable program. Every tutorial in this book is a file in the repository's [tutorials/](#) folder.

Goal: Store key-value pairs and unique collections.

Dict: {"key": value}, read with `d["key"]`, write with `d["key"] = value`.

Set: {1, 2, 3}. Membership: `value of set`. Sets have no duplicates.

```
person = {"name": "Alex", "age": 7, "city": "Madurai"}
say("name: " + person["name"])
say("age: " + str(person["age"]))
person.update({"hobby": "coding"})
say("has hobby: " + str("hobby" of person))
say("hobby: " + person["hobby"])
say("keys: " + str(person.keys()))
say("get city: " + person.get("city"))
say("get missing (default void): " + str(person.get("missing")))
say("length: " + str(len(person)))

for key of person
    say(" " + key + " = " + str(person[key]))
end

primes = {2, 3, 5, 7, 11}
say("7 in primes: " + str(7 of primes))
say("8 in primes: " + str(8 of primes))
say("set size: " + str(len(primes)))

a = {1, 2, 3, 4}
b = {3, 4, 5}
say("a = " + str(list(a)))
say("a | b = " + str(list(a | b)))
say("a & b = " + str(list(a & b)))
say("a - b = " + str(list(a - b)))

bring sets
say("union: " + str(list(sets.union(a, b))))
say("intersection: " + str(list(sets.intersection(a, b))))
say("difference: " + str(list(sets.difference(a, b))))
say("is subset: " + str(sets.is_subset({1, 2}, a)))

# Challenge: count how many unique letters are in your name using a set.
```

Tutorial 9: Classes and Inheritance

Source file: `tutorials/09_classes.js` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Bundle data and behavior with object-oriented programming.

class Name ... end, constructor is `init(self, ...)`, use `self`.

Create instances with `new`. Inherit with `extends`.

`super` only works inside a subclass method.

```

class Animal
  func init(self, name)
    self.name = name
  end

  func speak(self)
    say(self.name + " makes a sound")
  end
end

class Dog extends Animal
  func speak(self)
    super.speak(self)
    say(self.name + " barks!")
  end
end

class Puppy extends Dog
  func init(self, name)
    super.init(self, name)
    self.cute = true
  end

  func speak(self)
    super.speak(self)
    say("So cute!")
  end
end

a = new Animal("Generic")
a.speak()

d = new Dog("Rex")
d.speak()

p = new Puppy("Milo")
p.speak()
say("Milo is cute: " + str(p.cute))

# Challenge: add a Cat class with its own speak() and try it out.

```

Tutorial 10: Files and JSON

Source file: `tutorials/10_files_json.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Read and write files, and work with JSON data.

Built-ins: `write_file(path, text)`, `read_file(path)`.

The `json` scroll adds `parse/stringify`, the `fs` scroll adds file helpers.

```
write_file("_tutorial_notes.txt", "line one\nline two\nline three\n")
content = read_file("_tutorial_notes.txt")
say("read back:")
say(content)

bring fs
say("fs.exists: " + str(fs.exists("_tutorial_notes.txt")))
say("line count: " + str(fs.count_lines("_tutorial_notes.txt")))
say("first line: " + fs.first_line("_tutorial_notes.txt"))
say("last line: " + fs.last_line("_tutorial_notes.txt"))

bring json
data = json.parse('{"name": "JTS GO", "version": 1, "features": ["easy", "fast"]}')
say("name: " + data["name"])
say("version: " + str(data["version"]))
say("first feature: " + data["features"][0])
say("stringify: " + json.stringify({"a": 1, "b": [2, 3]}))

json.save("_tutorial_data.json", {"app": "demo", "count": 3})
loaded = json.load("_tutorial_data.json")
say("loaded app: " + loaded["app"])
say("loaded count: " + str(loaded["count"]))
say("pretty:")
say(json.pretty(loaded))

# The files created above (_tutorial_*.txt/_tutorial_data.json) are
# gitignored scratch files; delete them before moving on if you like.
```

Tutorial 11: Errors and try/catch/finally

Source file: `tutorials/11_errors.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Handle problems gracefully instead of crashing.

`throw "message"` raises an error.

`try ... catch e ... finally ... end` catches it and always runs cleanup.

```
func divide(a, b)
  if b == 0
    throw "cannot divide by zero"
  end
  return a / b
end

func require_positive(n)
  if n <= 0
    throw "expected a positive number, got " + str(n)
  end
  return n
end

try
  result = divide(10, 0)
  say("result: " + str(result))
catch err
  say("caught: " + err)
finally
  say("cleanup always runs")
end

say("still running after the error")

try
  say("10 / 2 = " + str(divide(10, 2)))
catch err
  say("should not reach here")
end

try
  require_positive(-3)
  say("never printed")
catch err
  say("caught: " + err)
end

# Challenge: wrap an ask() in try/catch to reject empty input.
```

Tutorial 12: Machine Learning and Web

Source file: `tutorials/12_ml_web.jts` — a complete, runnable program. Every tutorial in this book is a file in the repository's `tutorials/` folder.

Goal: Peek at JTS GO's built-in AI/ML and web features.

ML: `seed()`, `rand()`, `sigmoid()`, `mse()`, `matrix()`, `matmul()`, `relu()`.

Web: `http_server(port)`, `http_route(srv, method, path, body)`,

`http_start(srv)` -- the last one blocks and serves requests.

```
# --- machine Learning ---
seed(42)
say("sigmoid(0) = " + str(sigmoid(0)))
say("sigmoid(2) = " + str(sigmoid(2)))

actual = [0, 0, 1, 1]
predicted = [0.1, 0.4, 0.8, 0.9]
say("mse = " + str(mse(predicted, actual)))

a = matrix([[1, 2], [3, 4]])
b = matrix([[5, 6], [7, 8]])
say("matmul = " + str(matmul(a, b)))

say("rand in [0,1]: " + str(rand() >= 0 and rand() < 1))
say("randint(1, 6) = " + str(randint(1, 6)))

# --- web server (register routes, do not start) ---
srv = http_server(8080)
http_route(srv, "GET", "/", "<h1>Hello from JTS GO</h1>")
http_route(srv, "GET", "/api/time", '{"now": 2026}')
say("server on port 8080 with 2 routes registered")
say("call http_start(srv) to serve, or run examples/website/server.jts")

# Challenge: train sigmoid on [0,1] -> 0 and [1,0] -> 1, then print mse.
```